

0.86.1. L'alphabet en gras, avec poserS — de base, ailleurs

Le 0.86 cinq lignes plus bas : `poserS(i, 5, ALPHABET_GRAS[i])`.

1. Ce qu'il faut comprendre

C'est le 0.86, cinq lignes plus bas : la ligne de départ est 5 au lieu de 0. `poserS` passe tout seul à la ligne 6 après le T.

Un seul nombre change, dans `poserS` : c'est la ligne.

C'est la version de base. Le 0.86.2 met l'ancien et le nouvel alphabet l'un au-dessus de l'autre.

Les fonctions de cette leçon — et le tuto de chacune

- `#include <ALPHABET_GRAS>` → [0.85.1. ALPHABET GRAS — les lettres en gras](#)
- `#include <poserS>` → [0.9.3. La fonction poserS\(\) — poser, et passer à la ligne](#)

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **1396 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

```
démarrage      49 o ← la console, à l'allumage - la mise en route de la console
main()         80 o ← démarrage - ta fonction
VBlank         8 o ← la console, à chaque image - compte les images, 60 fois par seconde
AttendreImage  28 o ← image() - attend l'image suivante
AttendreVBlank 16 o ← démarrage - attend que l'écran soit libre
AdresseCase    10 o ← poser() - trouve l'adresse d'une case de l'écran
Diviser        17 o ← poser() - divise - le processeur ne sait pas le faire
Reste          10 o ← poser() - le reste d'une division
CopierTuiles   18 o ← démarrage - copie les lettres et les dessins en mémoire vidéo
EffacerCarte   14 o ← démarrage - vide les deux cartes de l'écran
Tuiles        1120 o ← CopierTuiles - la police entière, et 26 tuiles dessinées
ALPHABET_GRAS_0 26 o ← poser() - une table gravée
```

Lettres gravées : toutes (`poserS()` reçoit un numéro calculé)

Pas gravé (jamais appelé) : `AvancerAirs`, `AttendreAcces`, `AttendreFinVBlank`, `EffacerCases`, `EcrireTexte`, `EcrireNombre`, `Multiplier`, `DecalerGauche`, `DecalerDroite`, `RangerLutins`, `InstallerTransfert`, `Hasard`, `EffacerFond`, `EffacerPanneau`, `LireManette`, `DonneesTransfert`, `Notes`

Fonctions du langage appelées :

```
poser() × 1    55 o sur place, dans main() + AdresseCase (10 o), Diviser (17 o), Reste (10 o), ALPHABET_GRAS_0 (26 o)
image() × 1     3 o sur place, dans main() + AttendreImage (28 o)
poserS() × 1     0 o sur place, dans main()
```

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

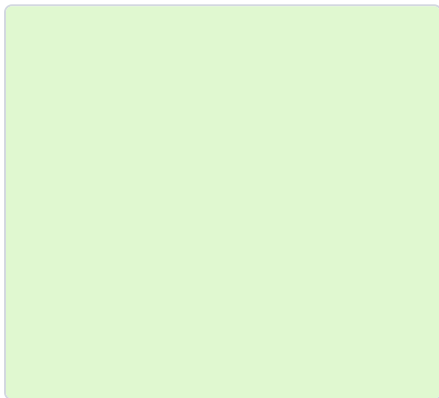
```
#include <ALPHABET_GRAS> // l'alphabet en gras : ALPHABET_GRAS[0] est le A gras
#include <poserS>         // pose une tuile, et passe à la ligne au bord
```

2. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

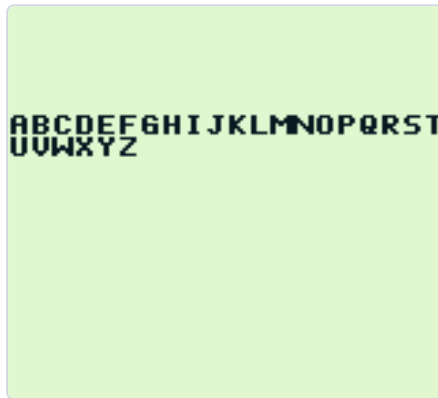
```
int main() {  
    // Le changement : la ligne, 0 → 5. U à Z iront sur la ligne 6.  
    for (uint8_t i = 0; i < sizeof(ALPHABET_GRAS); i++) {  
        poserS(i, 5, ALPHABET_GRAS[i]);  
    }  
  
    while (true) {  
        image();  
    }  
}
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20

4. Ce qu'on doit voir

L'alphabet en gras, sur les lignes 5 et 6.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

- ✓ 26 lettres grasses, toutes différentes (26)
- ✓ ce ne sont pas les lettres d'ALPHABET

Ensuite : **0.86.2. L'alphabet en gras, avec poserS — doublé, deux positions** — chapitre 0 : Avant tout. · gameboy3 — écrire une cartouche Game Boy en C++